Mathematics of Haptera Generation — Detailed Walk-Through

A thorough explanation of `haptera_export.py` building the trigonometry and vector calculus from first principles. Helical lanes and inter-haptera collision avoidance are omitted by request.

1. Cone geometry

1a. Trig refresher: the radial distance is the Pythagorean theorem

Any point (x, y, z) in 3D has a "horizontal radius" — its distance from the z-axis. If you stand looking down at the xy-plane from above, the point's shadow lands at (x, y), and that shadow's distance from the origin is

$$r_{\text{pos}} = \sqrt{x^2 + y^2}.$$

This is just Pythagoras applied to the right triangle with legs x and y. The z coordinate doesn't matter for this — it's a purely 2D measurement done in the xy-plane.

1b. Trig refresher: similar triangles for the cone wall

The cone has apex at the top ($z = H$) where the radius is 0, and base at the bottom ($z = 0$) where the radius is R. The wall is a straight line in any vertical cross-section. Pick any height z between them; what's the wall radius there?

Draw a vertical right triangle with the apex at top, height H down to the base, and horizontal leg R at the bottom. The wall is the hypotenuse. At height z, you're a distance $(H - z)$ below the apex. By similar triangles,

$$\frac{r_{\text{wall}}(z)}{H - z} = \frac{R}{H} \quad \Longrightarrow \quad r_{\text{wall}}(z) = \frac{R}{H}(H - z).$$

The factor R/H is the cone's *slope*: how much the radius grows per unit drop in z. Sanity check: at $z = H$, $r_{\text{wall}} = 0$ (apex); at $z = 0$, $r_{\text{wall}} = R$ (base).

1c. Why we subtract the tube radius

`cone_contains` doesn't just ask "is the skeleton point inside?" — it asks "does the *tube* fit?" A tube is a cylinder of radius r around the skeleton, so the tube's outer surface reaches r further from the centerline in any horizontal direction. To keep the tube surface inside the wall, the skeleton has to stay r further inside than the wall:

$$r_{\text{pos}} \le r_{\text{wall}}(z) - r.$$

That's all the function does.

1d. Vector-calculus refresher: the gradient as a perpendicular

This is the key tool for `inward_direction`. A *gradient* is the vector built from the partial derivatives of a function $f(x, y, z)$:

$$\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z} \right).$$

Two facts about gradients of *implicit surfaces* (surfaces defined by $f = 0$):

1. **The gradient at a point on the surface points perpendicular (normal) to the surface.** Intuition: the gradient points in the direction of fastest increase of f. If you walk *along* the surface, f stays at 0 — so the direction of fastest increase must be the direction that leaves the surface fastest, which is straight off it. 2. **The gradient points toward higher values of f.** So if $f > 0$ outside the surface and $f < 0$ inside, ∇f points *outward*.

1e. Applying it: the inward cone normal

Write the cone wall as the level set of an implicit function. The wall consists of points where the radial distance equals the wall radius:

$$r_{\text{pos}} = r_{\text{wall}}(z) \iff \sqrt{x^2 + y^2} - \frac{R}{H}(H - z) = 0.$$

Expanding: $\sqrt{x^2 + y^2} + \frac{R}{H} z - R = 0$. Let

$$f(x, y, z) = \sqrt{x^2 + y^2} + \frac{R}{H} z - R.$$

Outside the cone, $\sqrt{x^2 + y^2}$ is bigger than r_{wall}, so $f > 0$. Inside, $f < 0$. So ∇f points outward, and $-\nabla f$ points inward.

Now compute the partials. Using the chain rule on the square root:

$$\frac{\partial}{\partial x} \sqrt{x^2 + y^2} = \frac{x}{\sqrt{x^2 + y^2}} = \frac{x}{r}, \qquad \frac{\partial}{\partial y} \sqrt{x^2 + y^2} = \frac{y}{r}, \qquad \frac{\partial}{\partial z} \frac{Rz}{H} = \frac{R}{H}.$$

So

$$\nabla f = \left(\frac{x}{r},\ \frac{y}{r},\ \frac{R}{H} \right).$$

The inward direction is $-\nabla f$, then normalized to unit length. The magnitude is

$$\|\nabla f\| = \sqrt{\left(\frac{x}{r}\right)^2 + \left(\frac{y}{r}\right)^2 + \left(\frac{R}{H}\right)^2} = \sqrt{\frac{x^2+y^2}{r^2} + \left(\frac{R}{H}\right)^2} = \sqrt{1 + (R/H)^2},$$

since $x^2 + y^2 = r^2$ by construction. So

$$\hat{n}_{\text{in}} = \frac{-1}{\sqrt{1+(R/H)^2}} \left(\frac{x}{r},\ \frac{y}{r},\ \frac{R}{H} \right).$$

Geometric interpretation. The first two components $(x/r, y/r)$ form a unit vector in the xy-plane pointing *away* from the axis — so $-(x/r, y/r)$ points toward the axis. The third component $-R/H$ is negative, meaning the inward normal also slopes *downward*. That's because the cone narrows as you go up; "moving deeper inside the cone" from a point on the wall actually means moving slightly down as well as toward the axis. Without this z component, branches would get pushed straight sideways into thin air just below the apex.

2. Branching recursion — building φ

2a. Trig refresher: polar coordinates and atan2

A 2D unit vector can be written using its angle from the $+x$ axis:

$$(\cos\theta,\ \ \sin\theta).$$

Going the other way — given (x, y), what's θ? — naively you'd write $\theta = \arctan(y/x)$. But $\arctan$ has range $(-\pi/2, \pi/2)$ only; it can't tell $(1, 1)$ from $(-1, -1)$, which both give $y/x = 1$. The fix is `atan2(y, x)`: it looks at the *signs* of x and y separately and returns the correct angle in $(-\pi, \pi]$. Geometrically, it's "the angle a vector (x, y) makes with the $+x$ axis, measured CCW."

2b. Extracting the parent's azimuth

The parent segment ends with unit direction $\hat{d} = (d_x, d_y, d_z)$. Its *azimuth* is the angle its xy-projection makes with the $+x$ axis:

$$\varphi_{\text{base}} = \mathbf{atan2}(d_y, d_x).$$

This throws away the z component — we only care about the rotation around the vertical axis right now.

2c. Adding the per-level twist

The script computes

$$\varphi = \mathbf{atan2}(d_y, d_x) + \ell \cdot \tau,$$

where ℓ is the depth level (root = 0) and τ = `TORSION` is a constant in radians. This is just adding a number to an angle — geometrically, it rotates the reference frame for the children by an extra $\ell\tau$ at each level. Without this, every generation's children would fan out in roughly the same plane and you'd get coplanar starbursts.

2d. Vector refresher: building a 3D direction by lateral kick

Each child should leave the parent at some sideways angle. The recipe:

```
angle  = (2*pi*i / k) + phi + jitter
spread = 0.28 + small_jitter
ndx = dx + cos(angle) * spread
ndy = dy + sin(angle) * spread
ndz = dz + small_z_jitter
```

What this does, geometrically: $(\cos\theta, \sin\theta, 0)$ is a unit vector lying flat in the xy-plane at azimuth θ. Multiplying by `spread` shrinks/grows it. Adding it to the parent direction $\hat{d}$ produces

$$\vec{d}_{\text{child}} = \hat{d} + s \cdot (\cos\theta,\ \sin\theta,\ 0) + (\text{small } z \text{ jitter}).$$

Think of it as the parent's arrow with a sideways tug. Then renormalize:

$$\hat{d}_{\text{child}} = \frac{\vec{d}_{\text{child}}}{\|\vec{d}_{\text{child}}\|}.$$

Normalization just rescales a vector to unit length: $\hat{v} = \vec{v}/\|\vec{v}\|$. This preserves direction but forces magnitude $= 1$.

The $2\pi i/k$ term spreads the k children evenly around a full circle. Adding φ rotates the whole fan to align with the parent direction (so child 0 is "in front" relative to the parent's

orientation, plus the per-level twist). The jitter $\xi \in [-0.25, +0.25]$ rad breaks symmetry so the tree doesn't look mechanical.

3. Wall steering

3a. Refresher: linear interpolation, clamped

The line

```
t = min((prox - STEER_ONSET) / (1 - STEER_ONSET), 1.0)
```

is a "remap" — take a value `prox` on the interval $[\rho_0, 1]$ and stretch it to $[0, 1]$:

- When `prox = STEER_ONSET` (ρ_0), $t = 0$.
- When `prox = 1`, $t = 1$.
- When `prox > 1`, the `min` clamps t to 1.
- When `prox < STEER_ONSET`, the surrounding `if` skips the function entirely.

This is the standard "lerp" pattern for activation curves.

3b. The quadratic ramp $w = S \cdot t^2$

Multiplying by t^2 instead of t gives a **soft onset**: at $t = 0.1$, linear gives weight $0.1S$ but quadratic gives $0.01S$ — barely any push. Near $t = 1$, both are close to S. So the steering kicks in gently and ramps up sharply. This is purely an engineering choice; mathematically it's just $f(t) = t^2$ on $[0, 1]$.

3c. Vector refresher: blending two directions

Steering does

$$\hat{d}' = \mathbf{normalize}(\hat{d} + w\,\hat{n}_{\text{in}}).$$

To picture this: place arrow $\hat{d}$ at the origin, place arrow $w\hat{n}_{\text{in}}$ at the same origin. Add them tip-to-tail. The result is a new arrow somewhere between them, closer to whichever was longer. Then rescale to unit length.

When $w = 0$, the result is just $\hat{d}$ (no change). When w is huge, the result is essentially $\hat{n}_{\text{in}}$. For intermediate w, you get a smooth interpolation in direction. This is *not* the same as

rotating $\hat{d}$ toward $\hat{n}_{\text{in}}$ by a fixed angle — it's a vector-sum-and-normalize, which is computationally cheaper and qualitatively similar.

4. Tube mesh — Rodrigues rotation

This is the heaviest math in the file. The goal: take a cylinder built along the $+z$ axis and rotate it so its long axis aligns with the segment direction $\hat{d}$. We need a 3×3 rotation matrix R that maps $\hat{z} \to \hat{d}$.

4a. Vector refresher: the dot product

For any two vectors $\vec{a}, \vec{b}$:

$$\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y + a_z b_z = \|\vec{a}\| \|\vec{b}\| \cos\theta,$$

where θ is the angle between them. **For unit vectors**, $\|\vec{a}\| = \|\vec{b}\| = 1$, so

$$\hat{a} \cdot \hat{b} = \cos\theta.$$

So the dot product directly gives you the cosine of the angle. To get θ itself: $\theta = \arccos(\hat{a} \cdot \hat{b})$. The code uses this: `angle = np.arccos(np.dot(z_axis, direction))`.

4b. Vector refresher: the cross product

For 3D vectors:

$$\vec{a} \times \vec{b} = (a_y b_z - a_z b_y,\ a_z b_x - a_x b_z,\ a_x b_y - a_y b_x).$$

Three properties matter:

1. **It's perpendicular to both inputs**: $(\vec{a} \times \vec{b}) \cdot \vec{a} = 0$ and $(\vec{a} \times \vec{b}) \cdot \vec{b} = 0$. 2. **Its magnitude is** $\|\vec{a}\| \|\vec{b}\| \sin\theta$, where θ is the angle between them. 3. **Right-hand rule** for direction: if $\vec{a}$ points along your fingers and curls toward $\vec{b}$, $\vec{a} \times \vec{b}$ points along your thumb.

To rotate vector $\hat{z}$ onto vector $\hat{d}$, the rotation has to happen in the plane spanned by these two vectors, and the rotation axis is the perpendicular to that plane — exactly $\hat{z} \times \hat{d}$.

So the code does:

```
axis = np.cross(z_axis, direction)
```

and the magnitude $\|\hat{z} \times \hat{d}\| = \sin\theta$. The unit rotation axis is

$$\hat{a} = \frac{\hat{z} \times \hat{d}}{\|\hat{z} \times \hat{d}\|} = \frac{\hat{z} \times \hat{d}}{\sin\theta}.$$

4c. Why a special case when axis is zero

If $\hat{d}$ is exactly $+\hat{z}$, then $\hat{z} \times \hat{d} = 0$ (you can't rotate a vector to itself nontrivially, no rotation needed). If $\hat{d}$ is exactly $-\hat{z}$, then $\hat{z} \times \hat{d} = 0$ also (any horizontal axis would work; the formula doesn't pick one). The code's special case handles both: identity matrix for $+\hat{z}$, the matrix `diag(1, -1, -1)` (a 180° rotation about the x-axis) for $-\hat{z}$.

4d. Linear-algebra refresher: the cross-product matrix

For a unit axis $\hat{a} = (a_x, a_y, a_z)$, define the **skew-symmetric matrix**

$$[\hat{a}]_\times = \begin{pmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{pmatrix}.$$

The point of this matrix: for any vector $\vec{v}$, multiplying $[\hat{a}]_\times \vec{v}$ gives the same answer as the cross product $\hat{a} \times \vec{v}$. So this matrix *is* "cross with $\hat{a}$" expressed as a linear operator.

You can verify by direct multiplication: the x component of $[\hat{a}]_\times \vec{v}$ is $0 \cdot v_x + (-a_z) v_y + a_y v_z = a_y v_z - a_z v_y$, which matches the x component of $\hat{a} \times \vec{v}$.

4e. The Rodrigues formula

The 3×3 rotation matrix for rotation by angle θ around unit axis $\hat{a}$ is

$$R = I + (\sin\theta)\,[\hat{a}]_\times + (1 - \cos\theta)\,[\hat{a}]_\times^2.$$

Geometric derivation (the rusty-trig version). Take any vector $\vec{v}$. Split it into two pieces:

- $\vec{v}_\parallel = (\vec{v} \cdot \hat{a})\,\hat{a}$ — the component along the rotation axis. This part doesn't move when you rotate.
- $\vec{v}_\perp = \vec{v} - \vec{v}_\parallel$ — the component in the plane perpendicular to $\hat{a}$. This is what rotates.

Inside the rotation plane, you have $\vec{v}_\perp$. You also have a second vector perpendicular to it in the same plane: $\hat{a} \times \vec{v}$ (which equals $\hat{a} \times \vec{v}_\perp$ — the parallel part contributes zero to the cross product). So $\{\vec{v}_\perp,\ \hat{a} \times \vec{v}\}$ form a 2D basis for the rotation plane.

Rotating $\vec{v}_\perp$ by θ in this 2D basis (just like rotating a 2D vector):

$$\vec{v}'_\perp = \cos\theta\, \vec{v}_\perp + \sin\theta\, (\hat{a} \times \vec{v}).$$

Total rotated vector:

$$R\vec{v} = \vec{v}_\parallel + \vec{v}'_\perp = (\vec{v} \cdot \hat{a})\hat{a} + \cos\theta\, (\vec{v} - (\vec{v} \cdot \hat{a})\hat{a}) + \sin\theta\, (\hat{a} \times \vec{v}).$$

Group terms:

$$R\vec{v} = \cos\theta\, \vec{v} + (1 - \cos\theta)(\vec{v} \cdot \hat{a})\hat{a} + \sin\theta\, (\hat{a} \times \vec{v}).$$

Rewriting using matrix operators:

- $\cos\theta\, \vec{v} = \cos\theta \cdot I\vec{v}$
- $\sin\theta\, (\hat{a} \times \vec{v}) = \sin\theta\, [\hat{a}]_\times \vec{v}$
- $(\vec{v} \cdot \hat{a})\hat{a}$ — there's an identity: $[\hat{a}]_\times^2 \vec{v} = (\hat{a} \cdot \vec{v})\hat{a} - \vec{v}$ for unit $\hat{a}$. So $(\vec{v} \cdot \hat{a})\hat{a} = \vec{v} + [\hat{a}]_\times^2 \vec{v}$, and $(1 - \cos\theta)(\vec{v} \cdot \hat{a})\hat{a} = (1 - \cos\theta)\vec{v} + (1 - \cos\theta)[\hat{a}]_\times^2 \vec{v}$.

Add it all up:

$$\begin{aligned} R\vec{v} &= \cos\theta\, \vec{v} + (1 - \cos\theta)\vec{v} + (1 - \cos\theta)[\hat{a}]_\times^2 \vec{v} + \sin\theta\, [\hat{a}]_\times \vec{v} \\ &= \vec{v} + \sin\theta\, [\hat{a}]_\times \vec{v} + (1 - \cos\theta)[\hat{a}]_\times^2 \vec{v} \\ &= \big(I + \sin\theta\, [\hat{a}]_\times + (1 - \cos\theta)[\hat{a}]_\times^2\big)\vec{v}. \end{aligned}$$

That's the Rodrigues formula. The code expands the matrix entry-by-entry using $c = \cos\theta$, $s = \sin\theta$, $t = 1 - \cos\theta$:

$$R = \begin{pmatrix} ta_x^2 + c & ta_x a_y - sa_z & ta_x a_z + sa_y \\ ta_x a_y + sa_z & ta_y^2 + c & ta_y a_z - sa_x \\ ta_x a_z - sa_y & ta_y a_z + sa_x & ta_z^2 + c \end{pmatrix}.$$

Once you have R, the cylinder vertices get multiplied by R (rotating them onto the new axis) and translated to the segment's midpoint.

5. Volume convergence — the cubic correction

5a. Refresher: how volumes scale with linear dimensions

This is the master fact behind the model.

- **A cylinder** of radius r and length L: $V_{\text{cyl}} = \pi r^2 L$. If we keep L fixed and multiply r by f, the new volume is $\pi(fr)^2 L = f^2 \cdot \pi r^2 L$. **Scales as r^2.**
- **A hemisphere** of radius r: $V_{\text{hemi}} = \frac{2}{3}\pi r^3$. Scaling radius by f: $\frac{2}{3}\pi(fr)^3 = f^3 \cdot \frac{2}{3}\pi r^3$. **Scales as r^3.**

Why the different powers? A cylinder is essentially 2D radius × 1D length; only one dimension scales. A hemisphere is 3D radius scaling in all three spatial directions.

5b. Setting up the model

The "naive" total volume of all tubes (no overlap correction):

$$V_{\text{naive}} = \sum_i \pi r_i^2 L_i.$$

`naive_volume` computes this. Note: L_i stays fixed when we change radii — only r_i changes — so $V_{\text{naive}} \propto r^2$ when all radii scale together.

The "overlap" volume at junctions: at every shared endpoint, several tubes meet, and their flat caps double-count a hemisphere-sized blob. `overlap_volume` approximates this as $\frac{2}{3}\pi r^3$ per touching segment per junction. So $V_{\text{overlap}} \propto r^3$ when all radii scale together.

If we multiply every radius by f, the actual union volume should approximately follow

$$V(f) = f^2 V_{\text{naive}} - f^3 V_{\text{overlap}}.$$

To hit a target, we want to solve $V(f) = V_{\text{target}}$ for f.

5c. Calculus refresher: finding the maximum

$V(f)$ is a cubic polynomial. As f grows, the $-f^3 V_{\text{overlap}}$ term eventually dominates, so V goes back down. There's a maximum somewhere in between. To find it, set the derivative to zero:

$$V'(f) = 2f V_{\text{naive}} - 3f^2 V_{\text{overlap}}.$$

Factor: $f(2V_{\text{naive}} - 3f V_{\text{overlap}}) = 0$. Two solutions: $f = 0$ (the trivial one) and

$$f^* = \frac{2V_{\text{naive}}}{3V_{\text{overlap}}}.$$

Plug f^* back into V:

$$V_{\max} = V(f^*) = (f^*)^2 V_{\text{naive}} - (f^*)^3 V_{\text{overlap}}.$$

Some algebra:

$$V_{\max} = \left(\frac{2V_n}{3V_o}\right)^2 V_n - \left(\frac{2V_n}{3V_o}\right)^3 V_o = \frac{4V_n^3}{9V_o^2} - \frac{8V_n^3}{27V_o^2} = \frac{12V_n^3 - 8V_n^3}{27V_o^2} = \frac{4V_n^3}{27V_o^2}.$$

The code compares V_{target} to this maximum. If the target exceeds it, no value of f can reach it (you'd need to break the cubic model — say, change the tree topology) and the code falls back to the simple square-root scaling.

5d. Refresher: Newton's method

To find f such that $g(f) = 0$, Newton's method iterates

$$f_{\text{new}} = f_{\text{old}} - \frac{g(f_{\text{old}})}{g'(f_{\text{old}})}.$$

Geometric intuition: at f_{old}, draw the tangent line to the curve $y = g(f)$. The slope is $g'(f_{\text{old}})$. Where does that tangent cross zero? At $f_{\text{old}} - g(f_{\text{old}})/g'(f_{\text{old}})$. If g is well-behaved near a root, this lands much closer to the actual root, and you iterate until g is essentially zero.

Here, $g(f) = V_{\text{naive}} f^2 - V_{\text{overlap}} f^3 - V_{\text{target}}$, and $g'(f) = 2V_{\text{naive}} f - 3V_{\text{overlap}} f^2$. The code does up to 40 Newton steps with a stop-on-convergence check.

5e. The warm start

`f = np.sqrt(V_target / V_measured)`. This is the answer you'd get if there were no overlap term — i.e., if V scaled purely as r^2. From $V(f) = f^2 V_{\text{measured}}$ (assumption), $f^2 = V_{\text{target}}/V_{\text{measured}}$, so $f = \sqrt{V_{\text{target}}/V_{\text{measured}}}$. Starting Newton at this value usually converges in 2–5 iterations rather than 20+.

5f. Initial radius from a target volume

`build_segments` computes the starting radius:

$$r_0 = \sqrt{\frac{V_{\text{base}}}{N\pi L(D+1)}}.$$

This comes from inverting the cylinder formula. Approximate the tree as N roots, each with $(D+1)$ segments of length L along its main spine, giving total skeleton length $NL(D+1)$. Treat all radii as equal to r_0:

$$V_{\text{naive}} \approx \pi r_0^2 \cdot \big(NL(D+1)\big).$$

Solve for r_0:

$$r_0 = \sqrt{\frac{V_{\text{base}}}{N\pi L(D+1)}}.$$

This ignores the side branches, so it's a rough estimate — but the tree is rebuilt and then *rescaled* with `scale_radii(segs, sqrt(BASE_VOLUME / ref))` so the *actual* naive volume hits V_{base} before the iteration loop even starts. The initial r_0 just needs to be in the right ballpark to grow a tree without crashing into walls.

Recap of the trig/calc tools used

- **Pythagorean theorem**: radial distance in the xy-plane.
- **Similar triangles**: cone wall radius at any height.
- **atan2**: extracting an azimuth angle from a 2D vector while preserving quadrant.
- **Polar form** $(\cos\theta, \sin\theta)$: building a 2D direction from an angle, used for child placement.
- **Vector addition + normalization**: blending one direction toward another (children, steering).
- **Gradient** ∇f: perpendicular-to-surface direction for an implicit cone wall.
- **Dot product** $\hat{a} \cdot \hat{b} = \cos\theta$: extracting the angle between two unit vectors.
- **Cross product** $\hat{a} \times \hat{b}$ with magnitude $\sin\theta$: building a perpendicular for the rotation axis.
- **Rotation matrix** via Rodrigues: closed-form 3×3 matrix for rotating around any axis by any angle.
- **Calculus 1** — derivatives, critical points, factoring polynomials: finding where the cubic $V(f)$ peaks.
- **Newton's method**: iterative root-finding for the converged radius scale factor.
- **Power scaling** $V \propto r^2$ vs r^3: why the volume model is a cubic in f, not a quadratic.